

App Marketplace Migration — the process at a glance

For every RocketRide app owner. **Marketplace launch: September 1, 2026.** Whatever state your app is in today, it moves through the same four stages below. Full step-by-step instructions live in the **app-development-docs** package (start at **00-INDEX.md**).

Standalone app

Built separately from the start — own repo, own backend, own auth & router.

saas-structure app

Built on shell-ui / the RR saas repo app structure (in-repo or older RR workspace).

Already on staging

Scaffolded and running against staging.rocketride.ai — verify, then finish stages 3–4.

0

Environment setup

Install the staging extension (remove ALL old ones — version numbers are inverted), connect to staging.rocketride.ai, configure the deploy target in `.env`, claim your org's developer id.

DOC 01
1

Development — restructure into the serverless architecture • due today 6:00 PM

Scaffold via `createApp` (never hand-create) · re-target UI to the vendored shell · delete your own backend — all logic becomes `.pipe` pipelines · config-as-data · crons become schedules.

DOCS 02 + 03

Custom nodes?

Any pipeline component missing from the staging catalog must ship as a **tested PR on rocketride-server** (`nodes/src/nodes/<name>` + tests in `nodes/test`) and be **approved & visible in the staging catalog** before any pipe references it. Raise the PR today — reviews take time.

DOC 04
2

Move to staging

Validate every pipe against staging · deploy with `deployTo` (team-bound) · set schedules · `verifyApp` → `addApp` (server builds) · publish @me.

DOC 03 §6–9
3

Data migration — onto the staging DBs

Personal & open-source DBs are retired. All data moves to the staging-managed stores via the `rocketride_graph` / `rocketride_sql` / `rocketride_vector` nodes. Probe first, import through a pipe, then remove the old credentials.

DOC 05
4

Ready to deploy — final sign-off

Full checklist: build green, manifest complete, secrets server-side, data on staging stores, smoke pass in the shell from the app switcher. Send the completed checklist to Shashidhar.

DOC 06

MARKETPLACE LAUNCH — SEPTEMBER 1, 2026

🔖 Free app-building tokens: redeem coupon code **HACKANAPP** on staging (Account → Billing → redeem code) **before you start building** — it covers your migration's pipeline runs.

What each stage means for you

0

Environment setup **DOC 01** · ~30 min

Remove **every** existing RocketRide extension (identify by content, not version — 1.3.0-universal is the OLD build), fully quit the editor, install the VSIX from staging.rocketride.ai/client/vscode (in Cursor: inside Cursor), connect in Cloud mode, verify the 10-doc set, put all four **ROCKETRIDE_*** connection vars in **.env**, and claim your org's developer id on the Deploy tab — without it, Store and Deploy are read-only. Then redeem coupon code **HACKANAPP** (Account → Billing) for your app-building token grant.

1

Development — the serverless structure **DOCS 02 + 03** · due Aug 27, 6:00 PM

Inventory your bindings (shell API, config, storage, backend, cron, secrets, identity), write a one-page migration contract, then scaffold fresh with **client.deploy.createApp** and port in gated phases: domain logic → data layer → UI → polish. Your own backend disappears: routes and workers become **.pipe** files; auth becomes the shell's; hardcoded org vocabulary becomes editable data. Gate every phase on typecheck + tests + production build.

2

Move to staging **DOC 03 §6–9**

One generator owns all pipes (stable project ids, app-local copies bundled). **validate** every variant, deploy with **deployTo**, preview + set schedules with a TTL, then **verifyApp** → **addApp** → **publish @me**. Secrets move to the server-side environment as **\${ROCKETRIDE_*}** — nothing secret ever ships in the bundle or a pipe file.

3

Data migration to the staging DBs **DOC 05**

Probe the staging store with a throwaway pipe, generate your pipes in a **rocketride_*** default variant (personal-DB variant kept only as a generator-side fallback), export your old data, import it **through a deployed pipe or your seed path**, verify counts and read paths, then remove the personal/open-source DB credentials everywhere.

4

Ready to deploy **DOC 06**

Run the full per-app checklist (environment, structure, nodes, staging, data, smoke pass) and send it — completed or with blockers — to Shashidhar before September 1.

CUSTOM NODES — THE PR GATE

A custom node is **platform code**: it ships only through a reviewed PR on **rocketride-server** (implementation under **nodes/src/nodes/<name>**, tests under **nodes/test**, config schema, no secrets). It's usable on the cloud only once merged and visible in staging's services catalog. First check you actually need one — the catalog, **tool_python**, or **tool_http_request** cover most cases. Raise PRs today: they carry review latency.

WHY YOUR PERSONAL DB MUST GO

Marketplace apps can't depend on a database only you can reach. The staging DB nodes are identity-scoped **server-side by design** — the engine injects tenant identity through the DB broker, and no client-side workaround exists (proven exhaustively). Your deployed pipes get the store for free; your Aura/Postgres/Qdrant credentials get retired.

Ground rules (they apply at every stage)

- **Never hand-create app files** — apps come from the scaffold (**createApp** / New App wizard) only.
- **pnpm, not npm** · vendored **rocketride/shell** tarballs only, never the npm registry.
- **Never invent APIs** — verify every client method and pipeline component against the vendored client, the catalog, and the schemas.
- **Lifecycle verbs** (deploy / publish / schedule) run only against the DEPLOY connection pair.
- **Read the platform docs first** — **ROCKETRIDE_README.md** and its task router, before writing any RocketRide code.

NEED HELP WITH...

CONTACT

Anything app-builder related — app structure, scaffold, shell API, migration phases, pipelines, custom-node process, this guide

Shashidhar Babu
shashidhar.babu@rocketride.ai

Deployment & staging infrastructure — staging health, server-side build failures, DB broker, release train (Send to Stage / Deploy Everything)

Dmitrii